Chapter 1

Wiring Diagram Explained (ESP32_BT_Controller Reference)

This chapter explains the wiring diagram in Figure **??** and connects each pin group to how the **MICSBOL/ESP32_BT_Controller** firmware uses it.

The pin mappings referenced here come from `pins.h`. Runtime behavior is implemented mainly in:

- `control.cpp` (maps received controller state to actions)
- `hal_output.cpp` (PWM/GPIO output implementation)
- `input_hw.cpp` (ADC and indicator input reads, used for telemetry sources)
- `pulse.cpp` (non-blocking momentary outputs)

1.1 Diagram Scope and Project Mode

The repository provides at least two major wiring modes in `pins.h`:

- `FULL_RC_MODE`: direct ESP32 GPIO for switches and indicators.
- `FULL_RC_MODE_MCP`: adds an MCP23017 over I^2C for expanded digital I/O.

Important consistency note Your diagram labels analog inputs as `NUM1 (32)`, `NUM2 (33)`, `AIN1 (34)`, `AIN2 (35)`. That matches the repository `FULL_RC_MODE_MCP` analog mapping:

```
PIN_NUMERIC1 = 32
PIN_NUMERIC2 = 33
PIN_ANALOG_IND1 = 34
PIN_ANALOG_IND2 = 35
```

In contrast, `FULL_RC_MODE` uses:

```
PIN_NUMERIC1 = 34
PIN_NUMERIC2 = 35
PIN_ANALOG_IND1 = 36
PIN_ANALOG_IND2 = 39
```

So, the diagram is best interpreted as a **FULL_RC_MODE_MCP-style analog layout** with **FULL_RC_MODE-style indicator labels**. If you want a single-mode diagram, update the GPIO numbers to match the chosen mode.

1.2 The Diagram

Figure 1.1: Suggested ESP32 pin assignment and peripheral connections used by the controller firmware.

1.3 Left Side: Inputs (Telemetry and Status)

1.3.1 I^2C bus: SDA=21, SCL=22

In `pins.h`, both modes define:

```
PIN_I2C_SDA = 21
PIN_I2C_SCL = 22
```

Your diagram includes 4.7kΩ pull-up resistors to 3.3V, which is the standard I^2C wiring requirement for most boards (often they are also present on sensor modules).

How the repository uses I^2C

- In `FULL_RC_MODE`, I^2C is marked "reserved" in `pins.h`.
- In `FULL_RC_MODE_MCP`, I^2C is used for the MCP23017 expander (address `0x20`).

This means the I^2C wiring in the diagram supports either: sensors (telemetry sources) or the expander.

1.3.2 Analog inputs: NUM1/NUM2/AIN1/AIN2

The firmware reads analog channels using `analogRead(...)` in `input_hw.cpp`. Those readings are mapped into different telemetry formats, for example:

- plot channels: mapped to 0–255
- indicator analog/battery: mapped to 0–100

- numeric panels: scaled up to 16-bit range (0–65535)

In your diagram, these pins are used by example circuits:

- battery monitor via resistor divider (safe ADC scaling)
- LM35 temperature sensor (analog voltage output)
- LDR divider (light intensity)
- potentiometer (manual analog control / calibration)

Why the repo mentions "ADC1 only" In `pins.h`, analog inputs are intentionally assigned to ADC1-capable pins because ADC2 can be problematic when Bluetooth/Wi-Fi is active on ESP32. That's why the repo comments emphasize ADC1 safety.

1.3.3 Digital indicators: IND1..IND4

In `FULL_RC_MODE`, `pins.h` defines:

```
PIN_IND1 = 4
PIN_IND2 = 5
PIN_IND3 = 2
PIN_IND4 = 0
```

In the repository, indicator inputs are used to build a bit mask (see `input_hw.cpp`), which is then sent as telemetry so the Android UI can display multiple on/off status flags.

Boot strap caution GPIO0 and GPIO2 can affect ESP32 boot behavior. The repository explicitly warns for GPIO0:

```
PIN_IND4 0   // careful: avoid pull-up at boot
```

So, when wiring switches/indicators on these pins, ensure the default state does not prevent normal boot.

1.4 Right Side: Outputs (Actuators Driven by the RC State)

1.4.1 PWM outputs: sticks and knobs

In both modes, `pins.h` defines six PWM outputs:

```
PIN_PWM_STICK_LX = 18
PIN_PWM_STICK_LY = 19
PIN_PWM_STICK_RX = 23
PIN_PWM_STICK_RY = 25
PIN_PWM_KNOB_L   = 26
PIN_PWM_KNOB_R   = 27
```

These are written in `hal_output.cpp` using `ledcWrite(...)`. The control values come from `control.cpp`, which reads `rcStatePacket` fields such as `leftStickX` and `leftKnob`.

Servos in the diagram The diagram shows servos connected to PWM pins plus 5V and ground. That matches the HAL usage:

- steering: `halSetSteering(...)` writes PWM for a steering servo
- camera pan: `halSetCameraPan(...)` writes PWM for a pan servo

1.4.2 Motor direction pins: MD_LEFT and MD_RIGHT

In `pins.h`:

```
PIN_MD_LEFT  = 12   // keep LOW at boot
PIN_MD_RIGHT = 15   // keep LOW at boot
```

In `hal_output.cpp`, each motor command sets a direction pin and computes a PWM target. A dead-band around the center stick stops the motor to prevent jitter.

The HAL also ramps motor speed (timer-driven update) for smoother acceleration and less current spike.

1.4.3 Switch outputs: SW1..SW4 (direct) or SW1..SW6 (MCP)

The control layer decodes a switch bit mask in `control.cpp` and calls:

```
halSetSwitch(index, state)
```

In `FULL_RC_MODE`, the HAL writes up to four direct GPIO pins (`PIN`$_SW1$..`PIN`$_SW4$).In`FULL_RC_MODE_MC`

`Button events are handled in control.cpp by controlHandleEvent(eventId), which triggers:`

```
halPulseSwitch(index)
```

`Pulses are implemented in pulse.cpp via pulseStart(...)  and pulseUpdate(), allowing momentary actions (e.g., horn, latch trigger, reset line) without blocking the main loop.`

1.5 How to Customize This Wiring Safely

When adapting the diagram to your own robot:

1. Decide the project mode (FULL_RC_MODE or FULL_RC_MODE_MCP).
2. Update the diagram GPIO numbers to exactly match pins.h for that mode.
3. Keep all analog telemetry inputs on ADC1-capable pins (as the repository does).
4. Avoid wiring defaults that force GPIO0/GPIO2 into boot modes.
5. If your motor driver requires a different interface (two PWM lines, enable pins, etc.), modify hal_output.cpp.

1.6 Summary

The diagram maps cleanly to the repository's code structure:

- **Inputs** (analog + digital indicators) are read in input_hw.cpp and can be forwarded as telemetry.
- **Outputs** (PWM, direction pins, switches, pulses) are driven by control.cpp through hal_output.cpp.
- **I^2C** provides an expansion path for sensors and the MCP23017 mode.